

An Agentic Orchestration Layer for Machine-Learning Gravitational-Wave Analysis: Typed Skill Contracts, Fail-Closed Execution and Calibrated Multi-Model Inference

FAN ZHANG¹

¹TBD

ABSTRACT

Machine-learning pipelines now cover the full low-latency gravitational-wave workflow: detection (Aframe), parameter estimation (AMPLFI), unmodelled anomaly search (GWAK) and noise subtraction (DeepClean). Composing them for a given scientific request still requires expert judgement about data access, model applicability, significance calibration and compute. We present ML4GW-AGENT, an orchestration layer in which a large language model or a deterministic planner turns a natural-language request into a directed acyclic graph of typed skills, each bound to a versioned contract (JSON-schema inputs and outputs, machine-checked preconditions, immutable model revisions) and executed by a fail-closed adapter that records every artifact by cryptographic hash. We evaluate the layer on three axes: planning quality on a benchmark of NNN prompts across three model backbones, including an adversarial suite that measures silent, plausible-but-wrong outcomes; scientific fidelity on the 90 events of GWTC-1, GWTC-2.1 and GWTC-3 processed end to end through Aframe, AMPLFI and GWAK on the LIGO Data Grid, compared with the catalog and with the published Aframe O3 search; and significance calibration from time-shifted backgrounds of NNN days (Aframe) and NNN days (GWAK) together with injection campaigns. TBD results. The code, contracts, calibration tables and run manifests are public.

Keywords: Gravitational waves — Neural networks — Astronomy software — Astronomy data analysis

1. INTRODUCTION

Machine learning now covers every stage of the low-latency gravitational-wave workflow. Aframe detects binary black hole mergers directly from strain with a median latency of about three seconds (?) and has run in the LIGO–Virgo–KAGRA (LVK) online analysis since August 2025; AMPLFI produces posterior samples and sky maps seconds later (?); GWAK searches the same data for unmodelled transients (?); DeepClean subtracts environmental noise couplings using auxiliary witness channels (?). Offline, machine-learning searches of the third observing run (O3) have reached the sensitivity of matched filtering for stellar-mass binaries and have started to report their own candidates (??).

What has not been automated is the judgement that composes these tools for a given scientific question: which data source can serve the request, whether a model’s training range and detector set apply, what threshold turns a network output into a significance statement, whether an expensive step is justified, and how every number in the answer can be traced back

to immutable inputs. This judgement is currently supplied by experts writing scripts, and it is exactly the kind of task that large language model (LLM) agents are being proposed for across the sciences. Agents have planned and executed chemistry (??), designed experimentally validated nanobodies (?), and in astronomy interpreted multi-band galaxy photometry through an existing modelling code (?) and driven cosmological parameter inference (?).

The same studies expose the central risk. In an evaluation of an astrophysical multi-agent system on eighteen tasks, the dominant failure was not a crash but “silent incorrect computation”: syntactically valid code producing plausible but wrong results, without self-diagnosis (?). A benchmark of twelve coding agents on gravitational-wave problems found metric misuse, constraint violations and fabricated results (?). For a field whose results are significance statements about rare events, an agent that occasionally invents a number is worse than no agent.

This paper presents ML4GW-AGENT, an orchestration layer built around that constraint. Its design princi-

ple is that the agent may decide *what* to run but never *whether a result is valid*: every capability is a skill with a machine-checked contract, every adapter fails closed, every model is pinned, every threshold comes from a recorded background study, and every artifact is hashed into a manifest. We evaluate the layer on three axes. Planning: a benchmark of NNN natural-language requests across three LLM backbones, including an adversarial suite that measures how often a plausible-but-wrong analysis would have executed with and without contracts (Section 3). Scientific fidelity: the 90 events of GWTC-1, GWTC-2.1 and GWTC-3 processed end to end through Aframe, AMPLFI and GWAK on the LIGO Data Grid, compared with the catalog and with the published Aframe O3 search (Section 4). Calibration: time-shifted backgrounds and injection campaigns for the shipped model revisions, and a witness-channel noise-subtraction route with a signal-preservation check (Section 5). Section 6 reports the measured cost, and Section 7 discusses what this design does and does not solve.

2. DESIGN

ML4GW-AGENT separates three concerns that are usually entangled in an analysis script: *what* a request means (planning), *whether* a step may run (contracts and preconditions), and *how* it runs (adapters and executors). Figure ?? summarises the data flow.

2.1. Skill contracts

Every capability the agent can schedule is a *skill* described by a versioned YAML manifest: a name (`aframe.detect`, `amplfi.pe`, `gwak.scan`, `deepclean.clean`, ...), a JSON-schema for inputs and for outputs with `additionalProperties: false`, a list of machine-checkable preconditions, output validations (fields that must exist, artifacts that must be on disk), resource estimates, a risk level with an approval flag, and the adapter that implements it. Model-bearing skills take an immutable `model_revision` (a git commit or a SHA-256 of the weights) and refuse `UNPINNED`. Contracts are the only interface the planner sees; the planner cannot invent parameters or skip a precondition, because plan validation rejects any task whose parameters fail the input schema or whose references point to outputs that no upstream task declares.

2.2. Planning

A *plan* is a directed acyclic graph of tasks with typed references (`${fetch_data.outputs.strain_artifact}`) and optional conditions evaluated at run time (`when: ${inspect_data.outputs.quality_passed} truthy`).

Two planners produce plans. The deterministic baseline planner maps a request to routes by lexical intent (detection, parameter estimation, anomaly scan, cleaning) and encodes the domain rules: AMPLFI is conditioned on an Aframe candidate; GWAK needs its own 4096 Hz fetch; DeepClean cleaning runs only after its applicability check reports **applicable**; an unbounded request (“scan all of O3”) is refused at planning time. The LLM planner asks a language model for a plan under a structured-output schema derived from the contracts, validates it, and falls back to the baseline with a recorded reason when validation or the API fails. Both produce the same plan object, so every downstream guarantee is independent of the planner.

2.3. Fail-closed adapters and provenance

An adapter is the code that runs a skill. Adapters never fabricate: when a model, a credential, a witness channel or a calibration is missing, the task fails with a reason and dependent tasks are skipped, not executed with defaults. Each run produces a manifest that records the plan, every task’s resolved parameters, outputs, validations, warnings, timings, the packages and model revisions used, and the SHA-256 of every artifact. Mock adapters exist for software testing and are labelled **simulated: true** in every output; no mock result can enter a scientific record.

2.4. Calibration as a first-class object

Detection statistics are only meaningful against a background. The layer ships calibration tables keyed by model revision: thresholds at named false alarm rates from time-shifted background studies, with the livetime that supports them, and a FAR curve so that any score can be reported as a measured background rate. The planner resolves a requested FAR to a threshold for the pinned revision; if none is measurable at that rate for that revision, the plan carries a warning and the adapter marks the flag as uncalibrated. Section 5 describes the studies.

2.5. Executors and cost policy

A plan runs locally, on an HTCondor pool (the LIGO Data Grid), or on a remote GPU host over SSH; a Kubernetes executor is registered as deferred. Before submission the runtime estimates CPU- and GPU-hours, memory and transfer volume from the contracts and the timings measured in earlier runs, checks them against a budget policy, and requires explicit authorisation above a configurable GPU-hour limit. Long GPS-time requests are split into overlapping segments, submitted as sub-plans and merged without duplicate candidates.

3. EVALUATION OF PLANNING

3.1. *Benchmark*

3.2. *Backbones*

3.3. *Silent-failure suite*

4. SCIENTIFIC VALIDATION ON GWTC EVENTS

4.1. *Setup on the LIGO Data Grid*

4.2. *Detection: comparison with the catalog and with the Aframe O3 search*

4.3. *Parameter estimation: AMPLFI versus GWTC posteriors*

4.4. *Anomaly scores*

5. SIGNIFICANCE CALIBRATION

5.1. *Time-shifted backgrounds*

5.2. *Injects*

5.3. *DeepClean applicability and signal preservation*

6. COST AND SCALABILITY

7. DISCUSSION

8. CONCLUSIONS

¹ TBD. This research has made use of data or software ob-
² tained from the Gravitational Wave Open Science Cen-
³ ter (gwosc.org), a service of the LIGO Scientific Collab-
⁴ oration, the Virgo Collaboration, and KAGRA.

Software: ml4gw-agent, Buoy, Aframe, AMPLFI, GWAK, DeepClean, ml4gw, gwpy, astropy, PyTorch, HTCondor